



Module 4 : Statistical and Neural NLP Models

Dr. Priya R L, Mrs. Sunita Suralkar

Faculty Incharge for NCMPE64 (Natural Language Processing & Generative AI)

Department of Computer Engineering

VES Institute of Technology, Mumbai

Agenda - Part : II

- **Networks for NLP**
 - LSTM
 - GRU
- **Word Embeddings:**
 - Word2Vec,
 - GloVe,
 - FastText

Long Short-Term Memory (LSTMs)

- It is a type of RNN, proposed as a **solution to the vanishing gradients problem**.
- On step t , there is a **hidden state**($h(t)$) and a **cell state**, ($c(t)$)
 - Both are **vectors length, n**
 - The **cell** stores **long-term information**
 - The LSTM can **erase, write and read information** from the cell
- The selection of which information is erased/written/read is controlled by three corresponding **gates** (**gates are calculated things whose values are probabilities**)
 - a. The **gates are also vectors length n**
 - b. On each timestep, each element of the gates can be **open(1), closed(0), or somewhere in-between**.
 - c. The gates are **dynamic**: their value is computed based on the current context

Long Short-Term Memory (LSTMs)

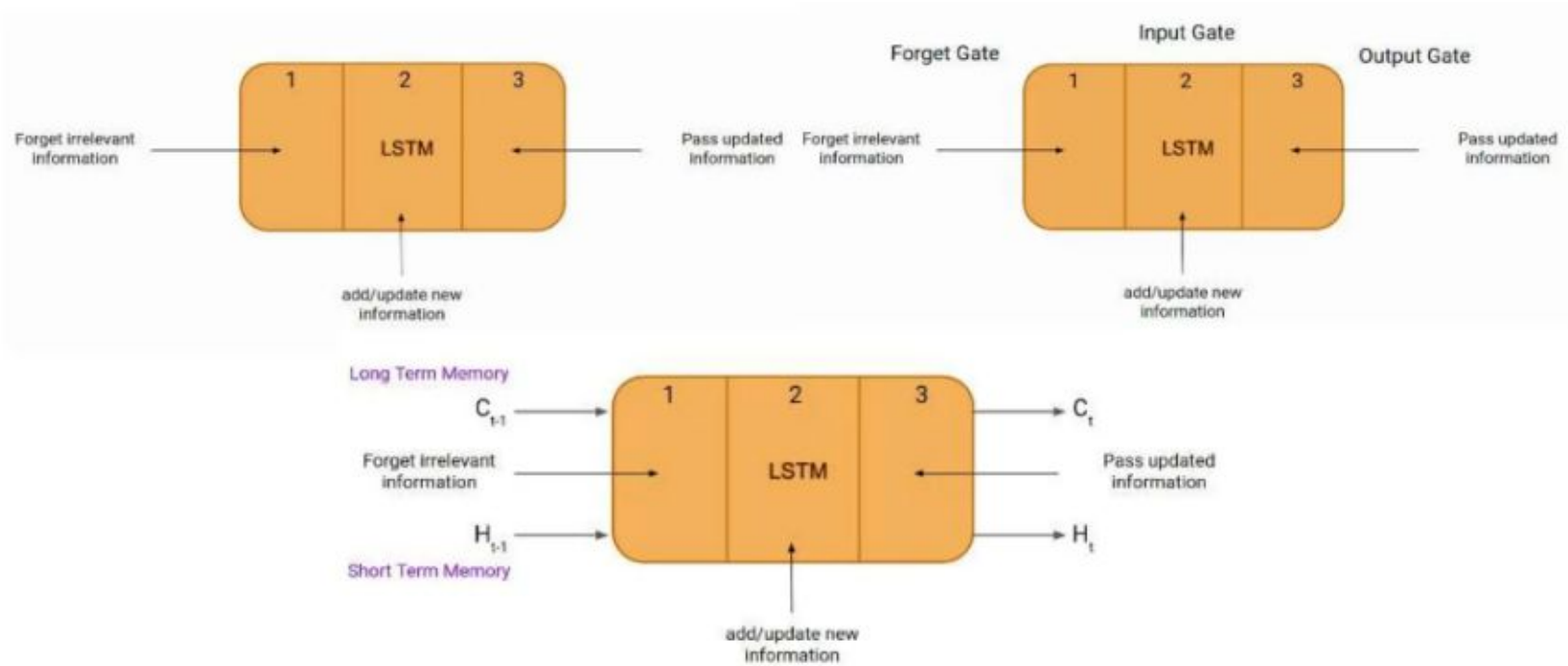
Solving Vanishing Gradients: LSTMs introduce a complex "cell state" that runs straight down the entire chain, acting like a conveyor belt for memory.

- The key component is a memory cell that acts like an accumulator (contains the identity relationship) over time

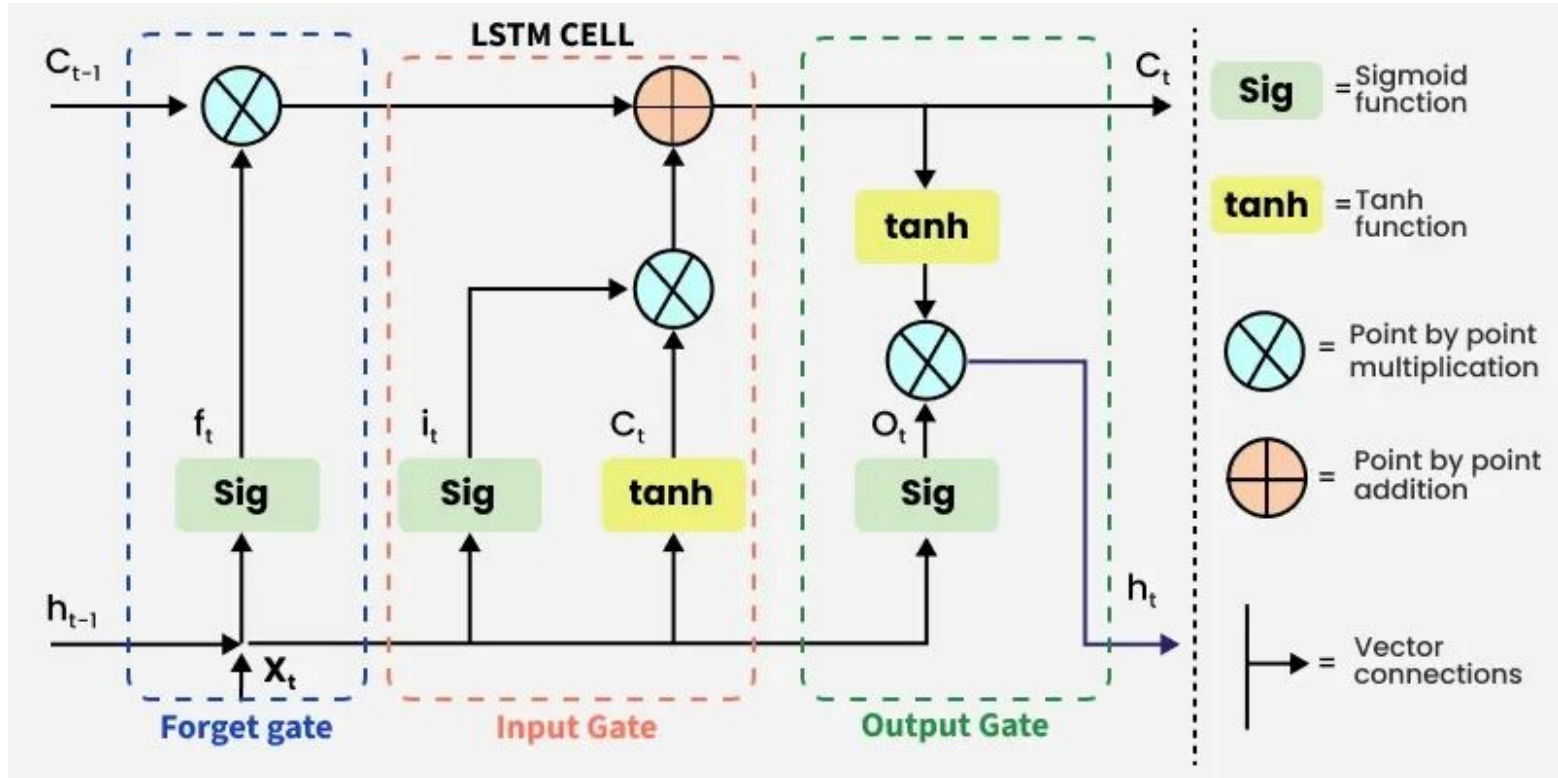
The Three Gates: LSTMs control information flow using specialized neural layers called gates:

- **Forget Gate:** Decides what information to throw away from the past state.
- **Input Gate:** Decides what new information to add to the cell state.
- **Output Gate:** Decides what parts of the cell state to output as the new hidden state

Long Short-Term Memory (LSTMs)



Long Short-Term Memory (LSTMs)



Long Short-Term Memory (LSTMs)

$$f^{(t)} = \sigma \left(W_f h^{(t-1)} + U_f x^{(t)} + b_f \right)$$

$$i^{(t)} = \sigma \left(W_i h^{(t-1)} + U_i x^{(t)} + b_i \right)$$

$$o^{(t)} = \sigma \left(W_o h^{(t-1)} + U_o x^{(t)} + b_o \right)$$

- LSTM does not guarantee that there is no vanishing / exploding gradient, but it does provide an easier way for the model to learn long-distance dependencies.

$$\tilde{c}^{(t)} = \tanh \left(W_c h^{(t-1)} + U_c x^{(t)} + b_c \right)$$

$$c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot \tilde{c}^{(t)}$$

$$h^{(t)} = o^{(t)} \odot \tanh c^{(t)}$$

Information Control: The Gates

- An LSTM cannot simply let all information onto the cell state, or it would fill up with garbage.
- LSTMs use "Gates" to optionally let information through.
- A gate is a neural network layer utilizing a sigmoid (σ) activation function and a pointwise multiplication operation.
- The sigmoid function outputs numbers strictly between 0 and 1.
- An output of 0 means "let nothing through" (forget completely).
- An output of 1 means "let everything through" (remember completely).

Step 1: The Forget Gate (f_t)

- The first step in an LSTM is deciding what information to throw away from the past.
- The Forget Gate looks at the previous hidden state (h_{t-1}) and the current word (x_t).
- It outputs a number between 0 and 1 for each number in the previous cell state (C_{t-1}).
- Equation:
$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$
- Example: For "The cat sat on the mat" a new subject is introduced in the sentence.

Step 2: The Input Gate (i_t)

- The second step is deciding what new information from the current word is worth storing.
- The Input Gate determines which specific values in the memory need to be updated.
- It uses a sigmoid function to act as a filter.
- Equation: $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$
- Like the Forget Gate, it looks at the previous hidden state and the current input word.

Step 3: Candidate Memory (C_t)

- Simultaneously with the Input Gate, the LSTM creates a vector of new candidate values.
- This is the actual new knowledge extracted from the current word.
- It uses a *tanh* activation function to squash values between -1 and 1 to regulate the network.
- Equation: $\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$
- This candidate memory is waiting to be filtered by the Input Gate.

Updating the Cell State (C_t)

- We multiply the old state (C_{t-1}) by the Forget Gate (f_t) to erase the old memories.
- We multiply the Candidate Memory by the Input Gate (i_t) to filter the new memories.
- Finally, we add these two components together.
- Equation: $C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$
- This is the crucial linear mathematical step that preserves gradients during training.

The Output Gate (o_t and h_t)

- The final step decides what the next short-term hidden state h_t will be.
- The short-term state is just a highly filtered version of the long-term cell state.
- The Output Gate uses a sigmoid function to decide what parts of the cell state to reveal.
- Equation for Output Valve:
$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$
- We then push the cell state through a $\tanh$ function and multiply it by the output valve.
- Final Equation:
$$h_t = o_t * \tanh(C_t)$$

Applications

- **Language Modeling:** Used in tasks like language modeling, machine translation and text summarization. These networks learn the dependencies between words in a sentence to generate coherent and grammatically correct sentences.
- **Speech Recognition:** Used in transcribing speech to text and recognizing spoken commands. By learning speech patterns they can match spoken words to corresponding text.
- **Time Series Forecasting:** Used for predicting stock prices, weather and energy consumption. They learn patterns in time series data to predict future events.
- **Anomaly Detection:** Used for detecting fraud or network intrusions. These networks can identify patterns in data that deviate drastically and flag them as potential anomalies.
- **Recommender Systems:** In recommendation tasks like suggesting movies, music and books. They learn user behavior patterns to provide personalized suggestions.
- **Video Analysis:** Applied in tasks such as object detection, activity recognition and action classification. When combined with Convolutional Neural Networks (CNNs) they help analyze video data and extract useful information.

Gated Recurrent Unit (GRU)

- **Gated Recurrent Unit (GRU)** - type of recurrent neural network designed to handle sequential data while reducing the complexity of traditional RNNs.
- GRUs are a simplified advancement of LSTM, where they merge multiple gates into update and reset gates, hence learning long-term dependencies with faster training and fewer parameters.
 - Simplified alternative to LSTM
 - Handles sequence and time-series data effectively
 - Widely used in NLP, speech and forecasting tasks

Gated Recurrent Unit (GRUs)

- Proposed by Cho et al. in 2014 as a **simpler alternative to the LSTM**.
- On each timestep t , we have input $x^{(t)}$ and hidden state $h^{(t)}$ (**no cell state**).

Update gate: controls what parts of hidden state are updated vs preserved

Reset gate: controls what parts of previous hidden state are used to compute new content

How does this solve vanishing gradient?
Like LSTM, GRU makes it easier to retain info long-term (e.g. by setting update gate to 0)

$$u^{(t)} = \sigma(W_u h^{(t-1)} + U_u x^{(t)} + b_u)$$

$$r^{(t)} = \sigma(W_r h^{(t-1)} + U_r x^{(t)} + b_r)$$

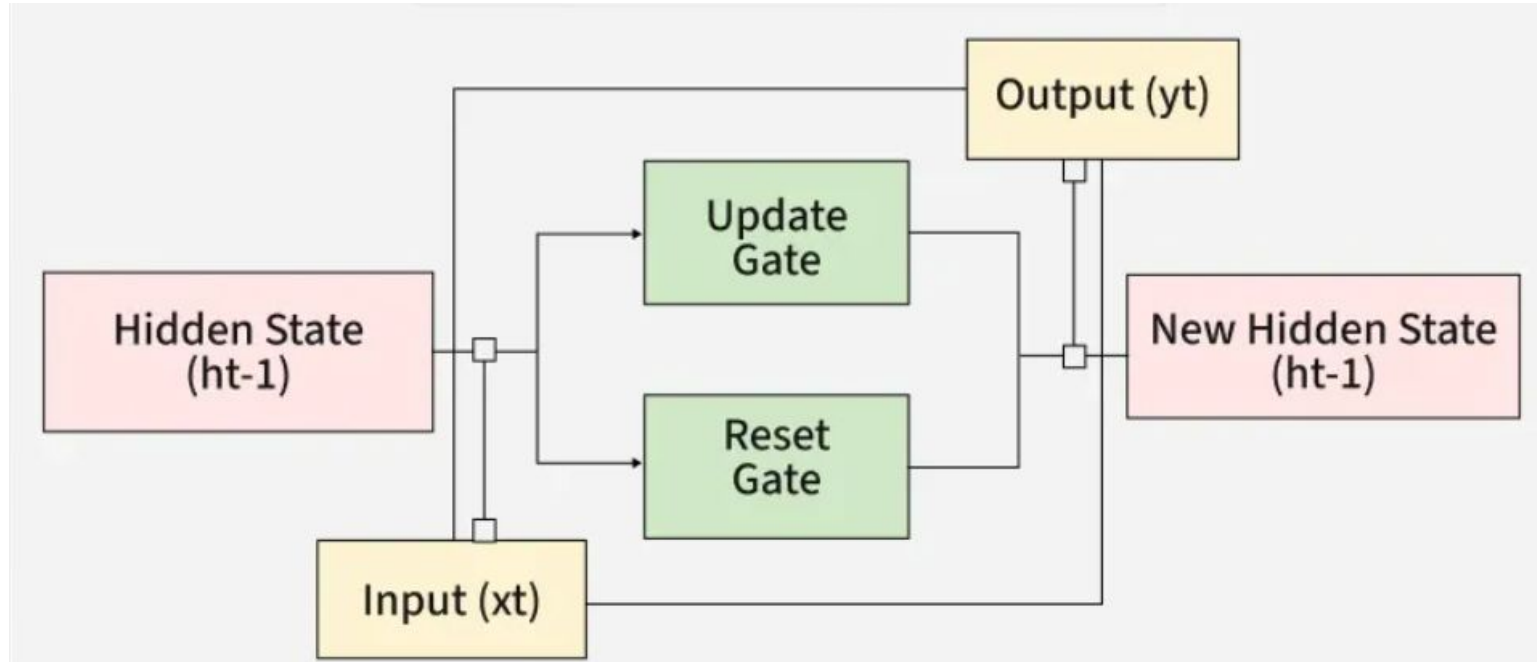
New hidden state content: reset gate selects useful parts of prev hidden state. Use this and current input to compute new hidden content.

Hidden state: update gate simultaneously controls what is kept from previous hidden state, and what is updated to new hidden state content

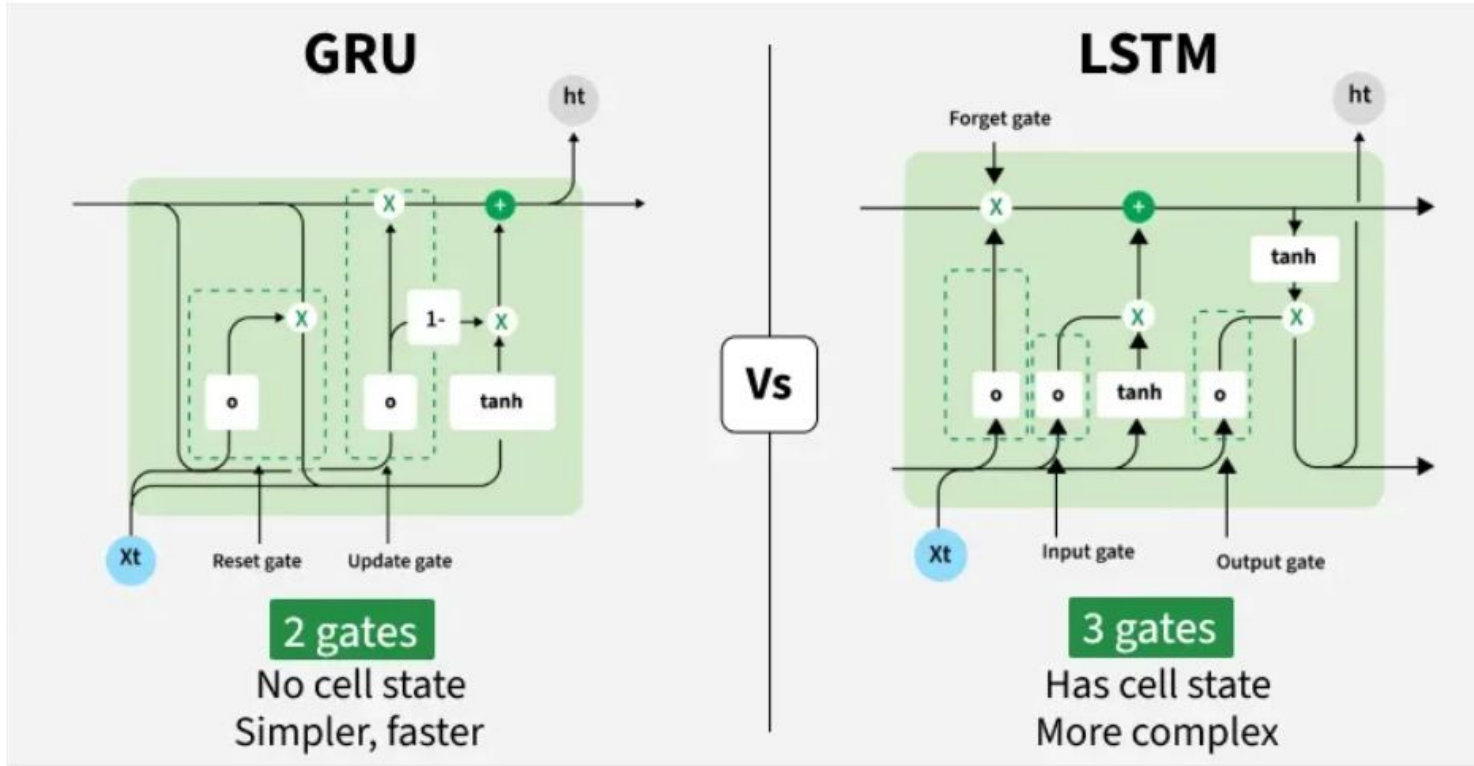
$$\tilde{h}^{(t)} = \tanh(W_h(r^{(t)} \circ h^{(t-1)}) + U_h x^{(t)} + b_h)$$

$$h^{(t)} = (1 - u^{(t)}) \circ h^{(t-1)} + u^{(t)} \circ \tilde{h}^{(t)}$$

GRU - How it Works?



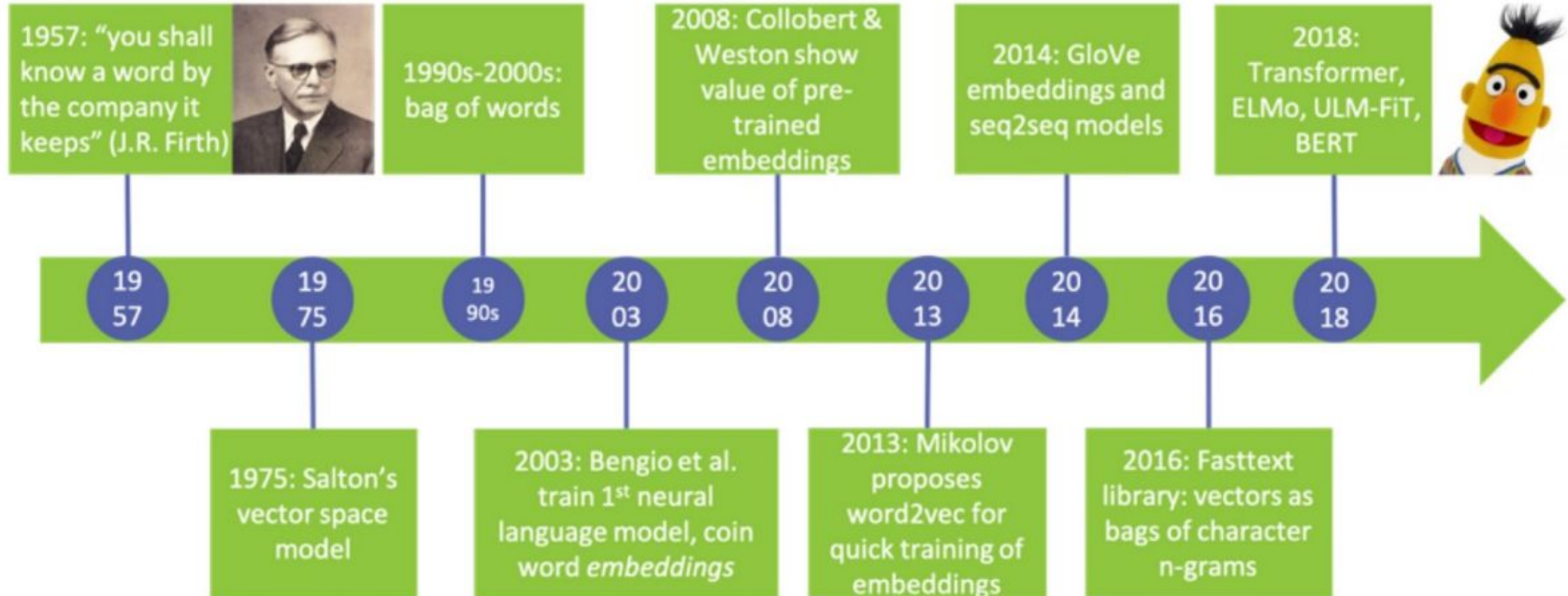
GRU - How it Works?



LSTM vs GRU

- Researchers have proposed many gated RNN variants, but LSTM and GRU are the most widely-used.
- The biggest difference is that GRU is quicker to compute and has fewer parameters.
- There is no conclusive evidence that one consistently performs better than the other.
- LSTM is a good default choice (especially if your data has particularly long dependencies, or you have lots of training data).
- **Rule of thumb:** start with LSTM, but switch to GRU if you want something more efficient.

Word Embeddings - Timeline



Representing Words as Discrete Symbols

In traditional NLP, we regard words as discrete symbols:

hotel, conference, motel – a **localist** representation

Means one 1, the rest 0s

Such symbols for words can be represented by **one-hot** vectors:

motel = [0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]

hotel = [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]

- Vector dimension = number of words in vocabulary (e.g., 500,000)

Representing Words by their Context

- **Distributional hypothesis:** words that occur in similar contexts tend to have similar meanings
- **J.R.Firth 1957**
 - “You shall know a word by the company it keeps”
 - One of the most successful ideas of modern statistical NLP!

*...government debt problems turning into **banking** crises as happened in 2009...*
*...saying that Europe needs unified **banking** regulation to replace the hodgepodge...*
*...India has just given its **banking** system a shot in the arm...*

Distributional Semantics



Context 1: A bottle of ____ is on the table.

Context 2: Everybody likes ____.

Context 3: Don't have ____ before you drive.

Context 4: We make ____ out of corn.

“tejuino” (in Spanish)- A cold beverage made from fermented corn and sweetened

Distributional Semantics

C1: A bottle of ____ is on the table.

C2: Everybody likes ____.

C3: Don't have ____ before you drive.

C4: We make ____ out of corn.

	C1	C2	C3	C4
tejuino	1	1	1	1
loud	0	0	0	0
motor-oil	1	0	0	0
tortillas	0	1	0	1
choices	0	1	0	0
wine	1	1	1	0

“words that occur in similar contexts tend to have similar meanings”

Count based Methods

The Term-Context matrix (or, word-word matrix)

- Each cell: number of times the row (target) word and the column (context) word co-occur in some context in the corpus
 - Generally, smaller contexts are used, like:
 - Paragraph
 - Window of 10 words
- **Each Word is a Vector.** Similar words are “nearby in space”
- **Word-Word Co-Occurrence Matrix:** use context vectors to represent the meaning of words!

	aardvark	computer	data	pinch	result	sugar	...
apricot	0	0	0	1	0	1	
pineapple	0	0	0	1	0	1	
digital	0	2	1	0	1	0	
information	0	1	6	0	4	0	

Count based Methods

- **The Term-Context matrix (or Word-word matrix)**
- Two **words are similar** in meaning if their context vectors are similar

	aardvark	computer	data	pinch	result	sugar
apricot	0	0	0	1	0	1
pineapple	0	0	0	1	0	1
digital	0	2	1	0	1	0
information	0	1	6	0	4	0

Count based Methods : Limitations

- **Metric size can increase** with an increase no. of vocabulary terms.
- **Vector size** will also be **very large** (if vocabulary has millions of words)
- **Sparse Matrix** = most of the entries in the matrix are zero
- If we have more **stop words, frequently appearing words like particles, prepositions**, then their corresponding entries will increase very high.

	aardvark	computer	data	pinch	result	sugar
apricot	0	0	0	1	0	1
pineapple	0	0	0	1	0	1
digital	0	2	1	0	1	0
information	0	1	6	0	4	0

Prediction based Methods

- **Word Embeddings:**

- Dense Vectors = most entries are non-zero
- Helps in learning less parameters
- May generalize better
- Can capture synonyms better
- Eg.: Car and Automobile are synonyms; but have distinct dimensions
- A word with car as a neighbour and a word with automobile as a neighbour should be similar. **but not..**

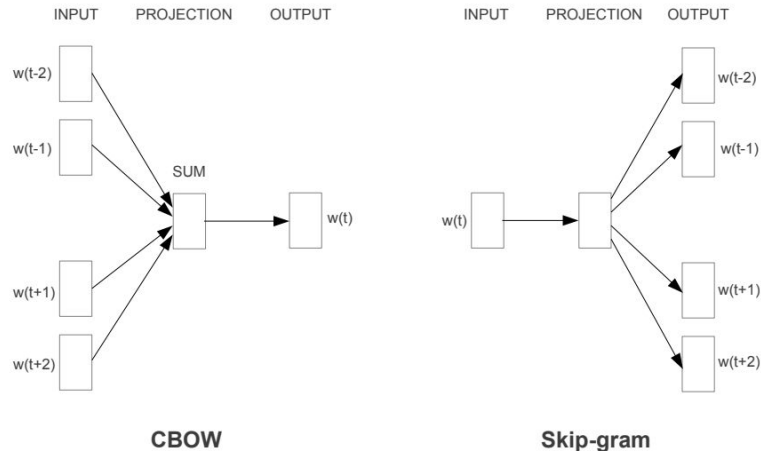
What are Word Embeddings?

- A word embedding is a **function that maps each word type to a single vector**.
- These vectors are typically **dense and have much lower dimensionality** than the size of the vocabulary.
- This mapping function typically ignores that the same string of letters may have **different senses** (**dining table vs. table of contents**) or **parts of speech** (**to table a motion vs. a table**)
- This mapping function typically **assumes a fixed size vocabulary**

Represent the Meaning of Word : Word2Vec

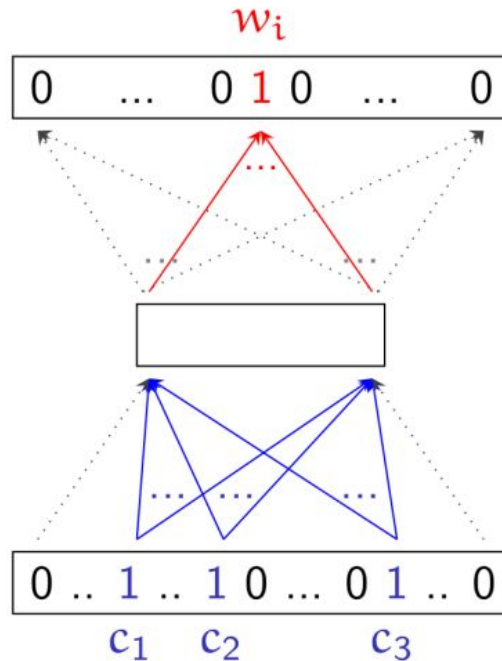
- **Two basic Neural Network Models:**

- **Continuous Bag of Words (CBOW):** use a window of word to predict the middle word. or Train a NN that given a set of context words, learns to predict the missing one.
- **Skip Gram (SG):** use a word to predict the surrounding words in window. or Train a NN that given a word, learns to predict likely words in its context.



Word2Vec (Mikolov 2013): CBOW

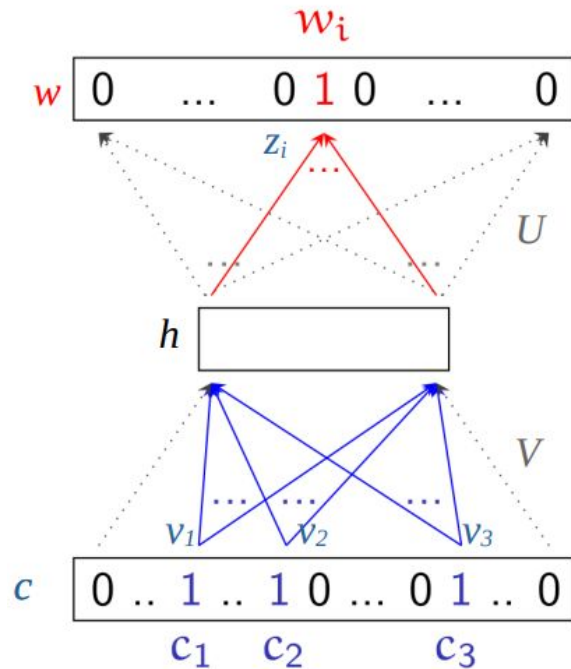
CBOW



- A dense low-rank vector representing a word is extracted from the **internal representation** of a Neural Network.
 - Word context words c_1 , c_2 , c_3 appearing around word w_i are used as a training example.
 - The NN is trained to predict the missing word for a given input context.
 - The hidden layer input weights encode the distribution of likely words in each context.
 - Words appearing in similar contexts will have similar hidden layer weights.
 - Hidden layer weights for each word are used as their dense vector representation

Word2Vec : CBOW

CBOW



- c : input vector, context words
 V : input layer weights
 h_c : hidden layer state for input c

$$h_c = V \cdot c = \sum_i V_i^T \cdot c = \sum_{i:c_i=1} V_i^T$$

- U : hidden layer weights
 z : output scores

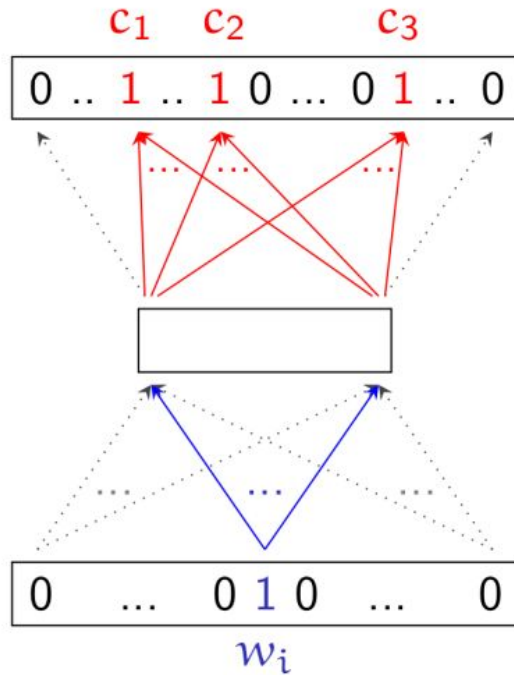
$$z_i = U_i \cdot h_c$$

- Softmax to convert scores to prob. distribution

$$w_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)} = \frac{\exp(U_i \cdot h_c)}{\sum_j \exp(U_j \cdot h_c)}$$

Word2Vec : Skip-Gram Model

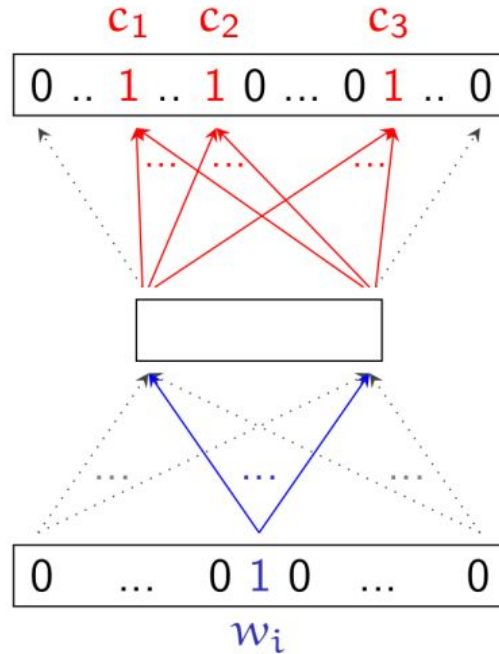
Skip-gram



- A dense low-rank vector representing a word is extracted from the **internal representation** of a Neural Network.
 - Word w_i appearing near context words c_1 , c_2 , c_3 is used as a training example.
 - The NN is trained to predict usual context words for a given input word.
 - The hidden layer input weights encode the usual contexts of each input word.
 - Word appearing in similar contexts will have similar hidden layer weights.
 - Hidden layer weights for each word are used as their dense vector representation

Word2Vec : Skip-Gram Model

Skip-gram



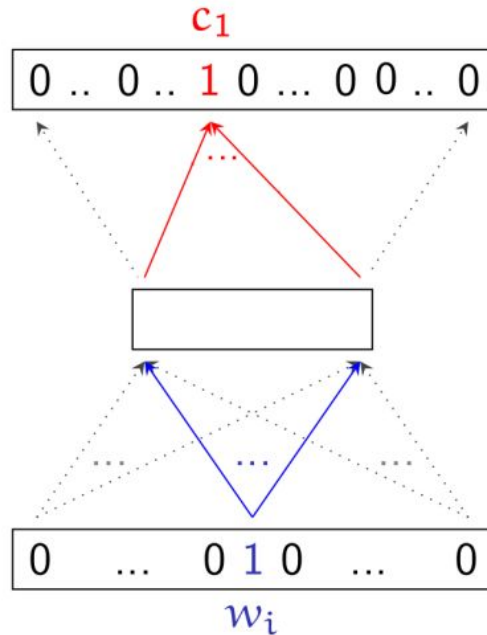
- **BUT:**

- The size of the vocabulary is usually large (e.g. 10,000 words)
- If the hidden layer has, e.g. 300 dimensions, that means $300 \times 10,000 \times 2$ parameters to update for each training example.

Too high computational cost

Word2Vec : Skip-Gram Model

Skip-gram



- We show the network only one target word at a time, so the amount of parameters to update is much smaller

But...

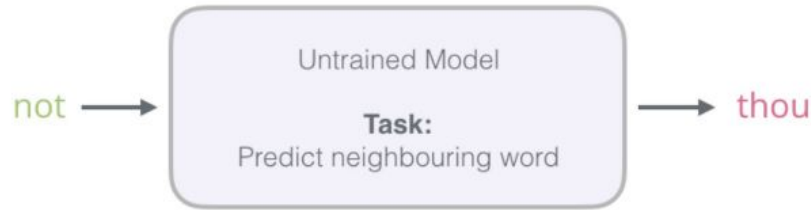
- If we use only positive examples, the network will overgeneralize and may end up assigning prob 1 to all words

We need to introduce negative examples

Skip-Gram with Negative Sampling

- We change the task to predict whether the words belong in the same context (0 or 1)

Change Task from



To:



Negative Examples

- This can be computed very efficiently – processing millions of examples in minutes.
- We can afford to introduce some negative examples at a reasonable cost

input word	target word
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine

input word	output word	target
not	thou	1
not	shalt	1
not	make	1
not	a	1
make	shalt	1
make	not	1
make	a	1
make	machine	1

Negative Examples

- We add **a few negative examples** for each positive pair in the dataset, with the same input word, a non-related context word, and a 0 label.

input word	output word	target
not	thou	1
not		0
not		0
not	shalt	1
not	make	1



Negative examples

We are contrasting the actual signal (positive examples of neighboring words) with noise (randomly selected words that are not neighbors), allowing the model to learn a trade-off.

Training Process

1) Pre-process the training text:

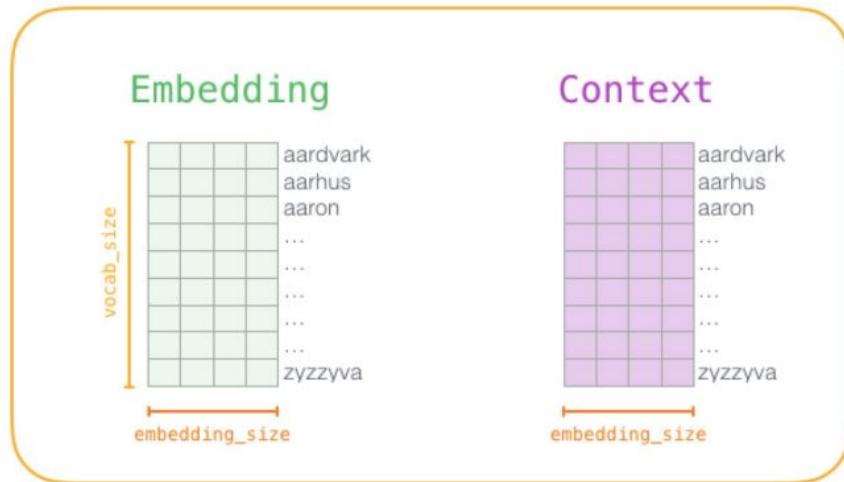
- Determine model vocabulary (which words are to be included in the model, and which will be the size of the model (number of vectors) $|V|$)
- Decide the dimension of the embedding E . Usual values are 100, 200, 300...

2) Create two matrices:

- Embedding matrix $|V| \times E$
- Context matrix: $|V| \times E$

Both matrices have an entry for each word in the vocabulary.

Both are initialized with random values.



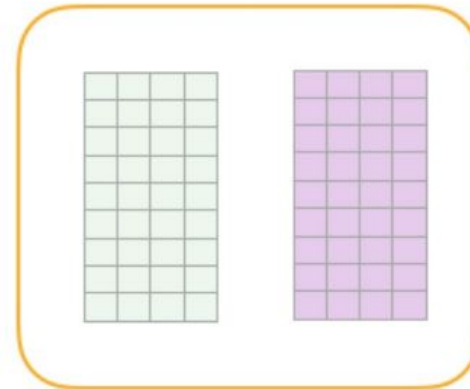
Training Process

- In each training step, we take one positive example and its associated negative examples.

dataset

input word	output word	target
not	thou	1
not	aaron	0
not	taco	0
not	shalt	1
not	mango	0
not	finglonger	0
not	make	1
not	plumbus	0
...	...	...

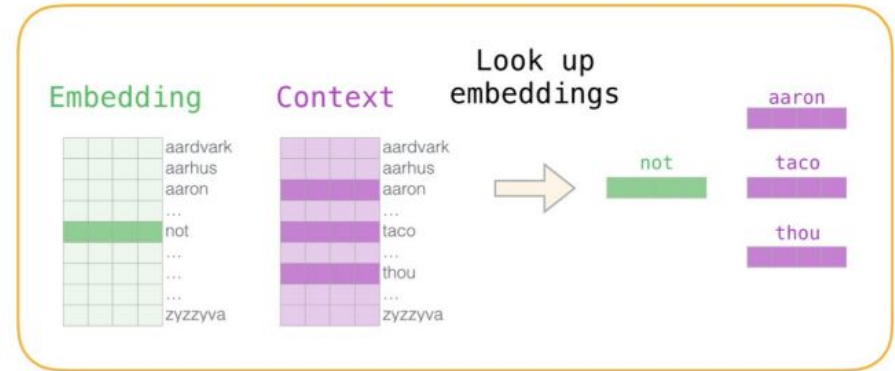
model



Training Process

- For each example, we have four words:
 - input word: *not*
 - output/context words:
 - thou* (actual neighbor)
 - aaron*, *taco* (negative examples).

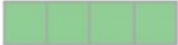
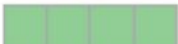
- Step 1: Look up current embeddings**
 - For input word, we look up in the Embedding matrix.
 - For context words, we look up in the Context matrix



Training Process

- **Step 2:**

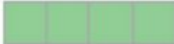
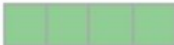
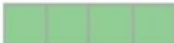
- Compute the **dot product** of the input word embedding with each of the context words embeddings, obtaining a similarity value
- Use sigmoid to convert these scores to probability-like values

input word	output word	target	input • output	sigmoid()
not 	thou 	1	0.2	0.55
not 	aaron 	0	-1.11	0.25
not 	taco 	0	0.74	0.68

Note that *taco* has the highest score and *aaron* still has the lowest score both before and after the sigmoid operations.

Training Process

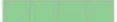
- **Step 3:**
 - Compute error in the model prediction.
(subtract the sigmoid scores from the target labels)

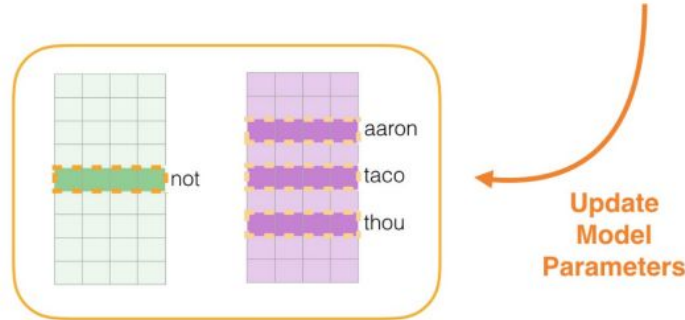
input word	output word	target	input • output	sigmoid()	Error
not 	thou 	1	0.2	0.55	0.45
not 	aaron 	0	-1.11	0.25	-0.25
not 	taco 	0	0.74	0.68	-0.68

Training Process

- **Step 4:** Update model.

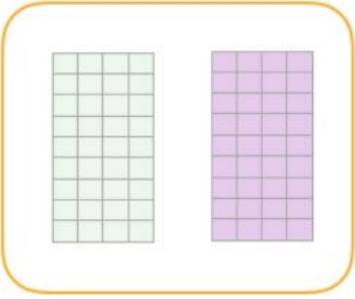
Use error score to **adjust the embeddings** of *not*, *thou*, *aaron*, and *taco* so that the next time the result would be closer to the target scores

input word	output word	target	input • output	sigmoid()	Error
not 	thou 	1	0.2	0.55	0.45
not 	aaron 	0	-1.11	0.25	-0.25
not 	taco 	0	0.74	0.68	-0.68



Training Process

- After processing this example, embeddings for the words involved in this step are slightly improved. We can proceed to the next example and repeat the process:

dataset			model	
input word	output word	target		
not	thou	1		
not	aaron	0		
not	taco	0		
not	shalt	1		
not	mango	0		
not	finglonger	0		
not	make	1		
not	plumbus	0		
...	...	...		

- Cycle through the entire dataset a number of times (or *epochs*).
- Finally, discard the Context matrix, and use the Embeddings matrix as our pre-trained embeddings model

Other Methods: Not Simply Words

- Instead of words, other units may be used:
 - **Phrases:** `Washington_Post` is a newspaper
Phrases can be generated using counts, e.g. $PMI = \log \frac{P(w_1, w_2)}{P(w_1) \cdot P(w_2)}$
 - **Characters:** `W a s h i n g t o n _ P o s t _ i s _ a _ n e w s p a p e r`
Create an embedding for each character, word representations are created combining its character vectors.
 - **Subwords:** `Wash #ing #ton Post is a new #spaper`
Different strategies to decide what subwords are used: n-grams, Byte Pair Encoding (BPE), Wordpiece, Sentencepiece, ...

Other Methods: fastText

- **fastText**: Subword (n -gram) based skip-gram.
 - Embedding vector for a word w is the sum of the embeddings of its n -grams ($3 \leq n \leq 6$).
 - e.g. The fastText representation of word *where* is the sum of 15 embedding vectors, for the n -grams:
 - 3-grams: *_wh*, *whe*, *her*, *ere*, *re_*
 - 4-grams: *_whe*, *wher*, *here*, *ere_*
 - 5-grams: *_wher*, *where*, *here_*
 - 6-grams: *_where*, *where_*
 - Whole word: *_where_*

Other Methods: GloVe

GloVe: Global Vectors for Word Representation.

- Ratios of word-word **co-occurrence probabilities** have the potential for encoding **semantic similarity**

$$P(j|i) = \frac{X_{ij}}{X_i}$$

X_{ij} : occurrences of word j the context of word i

X_i : occurrences of any word in the context of word i

Probability and Ratio	$k = solid$	$k = gas$	$k = water$	$k = fashion$
$P(k ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k ice)/P(k steam)$	8.9	8.5×10^{-2}	1.36	0.96

Other Methods: GloVe

GloVe: Global Vectors for Word Representation.

- **solid** has a much higher co-occurrence probability with **ice** than with **steam**

Probability and Ratio	$k = \text{solid}$	$k = \text{gas}$	$k = \text{water}$	$k = \text{fashion}$
$P(k \text{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \text{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k \text{ice})/P(k \text{steam})$	8.9	8.5×10^{-2}	1.36	0.96

Other Methods: GloVe

GloVe: Global Vectors for Word Representation.

- **gas** has a much higher co-occurrence probability with **steam** than with **ice**

Probability and Ratio	$k = solid$	$k = gas$	$k = water$	$k = fashion$
$P(k ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k ice)/P(k steam)$	8.9	8.5×10^{-2}	1.36	0.96

Other Methods: GloVe

GloVe: Global Vectors for Word Representation.

- **water** has a similar (high) co-occurrence probability with both **ice** and **steam**

Probability and Ratio	$k = solid$	$k = gas$	$k = water$	$k = fashion$
$P(k ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k ice)/P(k steam)$	8.9	8.5×10^{-2}	1.36	0.96

Other Methods: GloVe

GloVe: Global Vectors for Word Representation.

- *fashion* has a similar (low) co-occurrence probability with both *ice* and *steam*

Probability and Ratio	$k = solid$	$k = gas$	$k = water$	$k = fashion$
$P(k ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k ice)/P(k steam)$	8.9	8.5×10^{-2}	1.36	0.96

Other Methods: GloVe

GloVe: Global Vectors for Word Representation.

- Stochastic gradient descent (SGD) is used to minimize the squared error between the dot product of word vectors and the logarithm of their co-occurrence count.
- Once the model is trained, we have 2 vectors for each word
 - w_i : vector representing word i as the **center** word
 - $\tilde{w}_i$: vector representing word i as a **context** word
- Either one of them (or their sum $w_i + \tilde{w}_i$) are used as the word representation vector.

Advantages: GloVe

GloVe: Global Vectors for Word Representation.

- GloVe captures both local and global statistical information from text.
- More effective for analogy tasks due to its explicit modeling of word relationships.
- Often results in better embeddings for uncommon words compared to Word2Vec.